

11. Bölüm

Yapılar, Birlikler, Öznitelikler ve Yansıma

11.1 Yapılar

Yapılar, belleğin yönetimi ve verimlilik açısından hayati önem taşır. Sınıflar (**class**) ile olan benzerliği yeni başlayanları şaşırtsa da, arka plandaki çalışma mantıkları taban tabana zıttır. Yapılar aslında **değer tipi nesnelerdir**. Sınıflar "referans tipi" olup bellek tahsisi öbek (**heap**) bölgesinden yapılır. Yapılar "değer tipi" olup doğrudan yığın (**stack**) bölgesinde saklanırlar.

Yapılar ve Sınıflar

Yapıların (**struct**) sınıflardan en büyük farklarından biri kalıttır (**inheritance**). **Yapılar, başka bir yapı veya sınıftan türetilemez ve taban sınıf olamaz!** Yapılara sadece arayüzler (**interface**) uygulanabilir.

§11.1.1 Yapı Tanımlama ve Temel Yapı

Bir yapı (**struct**) tanımlamak, sınıf (**class**) tanımlamaya çok benzer ancak kullanım amacı genellikle küçük, hafif ve kısa ömürlü veri gruplarını temsil etmektir. Farklı tiplerdeki **değer tipi (value type)** elemanlarının bir arada gruplandırılan özel bir veri tipi tanımlamak için yapı tanımlarız. Yapılar, türetilmiş (**derived**) veya kullanıcı tanımlı (**user defined**) bir veri olup tanımlamak için **struct** anahtar kelimesini kullanırız.

```
public struct Nokta
{
    public int X { get; set; }
    public int Y { get; set; }
    public Nokta(int x, int y)
    {
        X = x;
        Y = y;
    }
    public void KomutYazdır() => Console.WriteLine($"Konum: ({X}, {Y})");
}
```

§11.1.2 Yapı Öğelerine Erişim

Tanımlanan yapı kullanılarak değişkenler aşağıdaki gibi tanımlanabilir;

```
yapı-kimliği değişken-kimliği;
//yada
yapı-kimliği değişken-kimliği=new yapı-kimliği();
```

Tanımlanan yapı değişkenleri üzerinden her bir üyeye **nokta işleci (dot operator)** ile erişilir. Bu işleç de parantez işleci ile aynı önceliktedir. Ayrıca **atama işleci (=)** bir yapıyı doğrudan kopyalamak için kullanılabilir. Bunun yanında, bir yapının üyesinin değerini başka birine atamak için yine atama işlecini de kullanabiliriz. Yukarıda örneği verilen yapılardan birçok değişken kimliklendirilebilir;

```
Öğrenci ilhan;
ilhan.yas = 20;
ilhan.cinsiyet = 'E';
ilhan.kilo = 80;
ilhan.boy = 180;
```

```
Öğrenci mehmet=new Öğrenci();
mehmet.yas = 25;
mehmet.cinsiyet = 'E';
mehmet.kilo = 75;
mehmet.boy = 175;
```

```
Koordinat nokta1;
nokta1.X = 10.5;
nokta1.Y = 20.5;

Koordinat nokta2= new Koordinat();
nokta2.X = 30.5;
nokta2.Y = 40.5;

nokta2 = nokta1;
```

§11.1.3 Yapılarda Yapıcılar

Sınıflardan ön tanımlı nesneler imal edilebildiği gibi yapılardan da varsayılan değerlere sahip yapılar oluşturulabilir. Bunun için **yapıcılar** (**constructor**) kullanılır. Yapı alanları (**field**) parametrelili bir yapıcı aracılığıyla imal edilir.

```
Koordinat nokta1;
nokta1.X = 10.5;
nokta1.Y = 20.5;
Console.WriteLine($"Nokta 1: ({nokta1.X}, {nokta1.Y})");
Koordinat nokta2 = new Koordinat();
nokta2.X = 30.5;
nokta2.Y = 40.5;
Console.WriteLine($"Nokta 2: ({nokta2.X}, {nokta2.Y})");
nokta2 = nokta1;
Console.WriteLine("Nokta 2: {0}, {1}", nokta2.X, nokta2.Y);
```

```
public struct Koordinat
{
    public double X;
    public double Y;
    public Koordinat()
    {
        X = 0;
        Y = 0;
    }
    public Koordinat(double x, double y)
    {
        X = x;
        Y = y;
    }
}
```

Mahrem (**private**) üyelere yalnızca yapıcı içinde ilk değer verilebilir yani başlatılabilir (**initialize**). Diğer değer tipleri gibi yapılar da **null** olamaz. Ancak değişken olarak **null** olabilir olarak tanımlanabilir;

```
Koordinat nokta1 = null; //HATA!: Değer tiplere null atanamaz!
Koordinat? v2 = null; //OK
Nullable<Koordinat> v3 = null // OK
```

§11.1.4 Yapılarda Ara Yüz Gerçekleştirme

Tanımlanan her yapı (**struct**), **System.ValueType** tipinden dolayı olarak miras alan kısır (**sealed**) bir veri tipidir. Yapılar başka herhangi bir tipten miras alamaz, ancak ara yüzleri gerçekleştirebilirler (**interface realization**);

```
Dikdörtgen dikdörtgen = new Dikdörtgen(5, 10);
Console.WriteLine($"Dikdörtgenin alanı: {dikdörtgen.Alan()}");
Daire daire = new Daire(7);
Console.WriteLine($"Dairenin alanı: {daire.Alan()}");
public interface IŞekil // Bir arayüz
{
    double Alan();
}
public struct Daire : IŞekil // Daire yapısı IŞekil arayüzünü gerçekleştiriyor
{
```

```

public double yaricap;
public Daire(double yaricap)
{
    this.yaricap = yaricap;
}
public double Alan()
{
    return Math.PI * yaricap * yaricap;
}
}
public struct Dikdörtgen : IŞekil // Dikdörtgen yapısı IŞekil ayayüzünü uyguluyor
{
    public double uzunKenar;
    public double kısaKenar;
    public Dikdörtgen(double uzunKenar, double kısaKenar)
    {
        this.uzunKenar = uzunKenar;
        this.kısaKenar = kısaKenar;
    }
    public readonly double Alan()
    {
        return uzunKenar * kısaKenar;
    }
}

```

Sınıflarda olduğu gibi bir yapıya üye yöntem tanımlanabilir. Eğer üye yöntem bir yapı alanını değiştirmeyecek ise yukarıdaki gibi `readonly` tanımlanabilir.

§11.1.5 Yapı ve Sınıf Arasındaki Farklar

Üç temel fark aşağıdaki tabloda verilmiştir;

Özellik	Struct (Yapı)	Class (Sınıf)
Bellek Bölgesi	Stack (Hızlı erişim)	Heap (Çöp toplayıcı yönetimi)
Tip Kategorisi	Değer Tipi (value type)	Referans Tipi (reference type)
Kalıtım	Kalıtımı desteklemez (Mühürlüdür)	Kalıtımı destekler

Tablo 11.1: Yapı ve Sınıf Karşılaştırması

§11.1.6 Değer Tipi Davranışı

Bir `struct` değişkenini başka birine eşitlediğinizde, verinin kendisi kopyalanır. Birinde yapılan değişiklik diğerini etkilemez.

```

Nokta n1 = new Nokta(10, 20);
Nokta n2 = n1; // n1'in kopyası oluşturuldu
n2.X = 100;
Console.WriteLine(n1.X); // Çıktı: 10 (Etkilenmedi!)
Console.WriteLine(n2.X); // Çıktı: 100

```

§11.1.7 Ne zaman Yapı Kullanmalı?

Aşağıda bu durumlar özetlenmiştir;

- ◊ **Boyut**: Eğer verinin boyutu 16 byte'tan küçükse genellikle `struct` uygundur.
- ◊ **Ömür**: Nesne çok kısa süre kullanılacak ve hemen yok edilecekse `struct` daha performanslıdır.
- ◊ **Değişmezlik (immutability)**: Yapıların `readonly` olarak tasarlanması en iyi uygulamadır.
- ◊ **Kalıtım Yokluğu**: Bir `struct` başka bir sınıftan veya yapıdan türeyemez. Ancak `interface` gerçekleştirebilir.

§11.1.8 Yalnız Okunabilir Modern Yapı

C#7.2+ ve sonrası için daha güvenli olan `readonly struct` yapısı kullanılır. Değiştirilemez yapılar, performans optimizasyonu sağlar.

```

public readonly struct Renk
{
    public int R { get; init; }
}

```

```

public int G { get; init; }
public int B { get; init; }

public Renk(int r, int g, int b) => (R, G, B) = (r, g, b);
}

```

Kutulama

Bir `struct` (değer tipi) nesnesini, `object` (referans tipi) bekleyen bir yönteme gönderirsek bellekte ne olur? Bu işleme kutulama (**boxing**) denir. Yığındaki (**stack**) veri, geçici olarak öbek (**heap**) bölgesinde bir kutuya alınır. Bu işlem performans açısından maliyetlidir. Bu yüzden döngüler içerisinde yapıları gereksiz yere `object` tipine dönüştürmekten kaçınmalıyız.

11.2 Birlikler

C# dünyasında, C++'taki `union` anahtar kelimesine doğrudan karşılık gelen bir yapı bulunmamaktadır. Ancak C#, bellek yönetimi üzerinde tam kontrol sağlamak istediğimiz durumlar için bu davranışı taklit etmemiz için `System.Runtime.InteropServices` kullanılır.

C++'ta `union`, içindeki tüm değişkenlerin aynı bellek adresini paylaşmasını sağlar. C#'ta bunu başarmak için `struct` kullanırız ve derleyiciye her bir değişkenin bellekte hangi "başlangıç noktası" (**offset**) değerinden başlayacağını el yordamıyla söyleriz.

Aşağıdaki örnekte, 4 adet 8-bitlik (**byte**) değer ile 1 adet 32-bitlik (**int**) değerın aynı bellek bölgesini nasıl paylaştığını görebilirsiniz;

```

using System.Runtime.InteropServices;
// FieldOffset(0) ifadesi, tüm değişkenlerin belleğin 0. noktasından başladığını belirtir (Union
↪ davranışı).
[StructLayout(LayoutKind.Explicit)]
public struct VeriPaketi
{
    [FieldOffset(0)]
    public int TamSayı; // 4 byte

    [FieldOffset(0)]
    public byte Bayt1; // TamSayı'nın 1. byte'ı
    [FieldOffset(1)]
    public byte Bayt2; // TamSayı'nın 2. byte'ı
    [FieldOffset(2)]
    public byte Bayt3; // TamSayı'nın 3. byte'ı
    [FieldOffset(3)]
    public byte Bayt4; // TamSayı'nın 4. byte'ı
}

```

Bu program için aşağıdakiler söylenebilir;

- ♦ **Aynı Adres, Farklı Bakış Açısı:** Yukarıdaki örnekte `TamSayı` değişkenine bir değer atadığınızda, aslında `Bayt1`'den `Bayt4`'e kadar olan kısımları da değiştirmiş olursunuz. Çünkü hepsi aynı fiziksel bellek hücrelerine bakar.
- ♦ **Kullanım Alanları:** Genellikle düşük seviyeli (**low-level**) ağ protokolleri, dosya formatları veya donanım sürücülerini çalışırken veriyi bit-bit parçalamak için kullanılır.
- ♦ **Güvenlik (safety):** Bu yapı C#'ın "güvenli" (**safe**) dünyasından biraz uzaklaştığı için dikkatli kullanılmalıdır. Yanlış ofset değerleri vermek veri bozulmalarına yol açabilir.

Bir başka örnek olarak bir IP adresini düşünebiliriz. Dahili olarak, bir IP adresi bir tam sayı (**int**) olarak tutulur. Ancak bazen `Bayt1.Bayt2.Bayt3.Bayt4` şeklinde farklı bayt öğelerine erişmek isteriz.

```

using System.Runtime.InteropServices;
var ip = new IPAdresi(new Random().Next());
Console.WriteLine($"{ip} = {ip.ipAdresi}"); //33.62.229.21 = 367345185
ip.Bayt1 = 100;
Console.WriteLine($"{ip} = {ip.ipAdresi}"); //100.62.229.21 = 367345252

[StructLayout(LayoutKind.Explicit)]
struct IPAdresi

```

```
{
    // "FieldOffset" ipAdresi tamsayısının adresidir.
    // sizeof(int) 4, sizeof(byte) = 1
    [FieldOffset(0)] public int ipAdresi;
    [FieldOffset(0)] public byte Bayt1; // Yapılın 1. byte'ı offset 0
    [FieldOffset(1)] public byte Bayt2; // Yapılın 2. byte'ı offset 1
    [FieldOffset(2)] public byte Bayt3; // Yapılın 2. byte'ı offset 2
    [FieldOffset(3)] public byte Bayt4; // Yapılın 2. byte'ı offset 3
    public IPAdresi(int address) : this()
    {
        ipAdresi = address;
    }
    public override string ToString() => $"{Bayt1}.{Bayt2}.{Bayt3}.{Bayt4}";
}
```

FieldOffset Kullanımı

Referans tiplerini (sınıflar, metinler vb.) **FieldOffset** kullanarak **aynı adrese çakıştıramazsınız!** Derleyici, bellek güvenliği nedeniyle buna izin vermez. Bu yapı sadece değer tipler (**value type**) olan **int**, **float**, **byte**, **struct** gibi tipler için çalışır.

11.3 record Anahtar Kelimesi

C# 9.0 ile hayatımıza giren **record** yapısı, nesne yönelimli programlamada "veriyi temsil etme" görevini sınıflardan devralır. Referans tipi olmasına rağmen değer bazlı karşılaştırma yeteneği sunar;

- Varsayılan olarak **init** property'leri ile gelir. Nesne bir kez oluşturulduktan sonra durumu değiştirilemez. Bu, çok kanallı (**multi-threaded**) ortamlarda güvenliği sağlar.
- ki farklı sınıf nesnesi içindeki veriler aynı olsa bile **==** operatörü ile kıyaslandığında false döner (referans farkından dolayı). İki **record** nesnesinin verileri aynıysa **true** döner
- Tek bir satırda koca bir veri modelini (Constructor, Finalizer/Deconstructor, Equality yöntemleri dahil) tanımlayabilirsiniz.

Bunu bir gerçek hayat örneği ile detaylandıralım. Banka dekontu üzerindeki bilgiler basıldıktan sonra değiştirilemez (**immutability**). Bir hata durumunda mevcut dekontu düzenlemek yerine, hatalı alanın düzeltildiği yeni bir kopyası oluşturulur (**with** ifadesi).

```
public record Kitap(string Isin, string Baslik);

var b1 = new Kitap("123", "C# Kitabı");
var b2 = new Kitap("123", "C# Kitabı");

// Referanslar farklı olsa da içerik aynı olduğu için True döner.
bool esit Mi = (b1 == b2);
```

11.4 Öznitelikler

C#'ta **öznitelikler** (**attributes**), bir programı oluşturan sınıf, yöntem, özellik gibi öğelere **bildirim detayını içeren bir bilgi eklemek için kullanılır**. Köşeli parantez [] içinde belirtilirler ve kodun çalışma mantığını değiştirmekten ziyade, o kodun nasıl ele alınması gerektiğini tanımlarlar.

§11.4.1 Hazır Öznitelikler

.NET kütüphanesi ile gelen ve sıkça kullanılan bazı **hazır öznitelikler** (**built-in attributes**) vardır:

- [**Obsolete**]: Bir yöntemin artık eskidiğini ve kullanılmaması gerektiğini belirtir. Derleyici bunu gördüğünde uyarı (**warning**) verir.
- [**Serializable**]: Bir sınıfın dosyaya veya ağa veya uzak bir ağa aktarılabilecek bir biçime dönüştürülebileceğini belirtir.
- [**Required**]: Özellikle ileride göreceğimiz ASP.NET ve Varlık çerçevesinde (**Entity Framework**) bir alanın (**field**) boş geçilemeyeceğini tanımlar.

```
public class EskiSistem
{
    [Obsolete("Bu yöntem verimsizdir, lütfen YeniSistem.YeniYöntem() kullanın.")]
    public void EskiYöntem() { /* ... */ }
```

}

§11.4.2 Kendi Özniteliğimizi Tanımlama

Kendi özniteliklerinizi (**custom attributes**) oluşturmak için `System.Attribute` sınıfından türetilen bir sınıf yazmanız yeterlidir.

[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method)] // Bu özniteliğin nerelerde
→ kullanılabileceğini kısıtlıyoruz.

```
public class BilgiAttribute : Attribute
{
    public string Yazar { get; }
    public string Versiyon { get; set; }

    public BilgiAttribute(string yazar)
    {
        Yazar = yazar;
    }
}
```

// Kullanımı:

```
[Bilgi("İlhan", Versiyon = "2.0")]
public class VeriIsleyici { }
```

§11.4.3 Çalışma Zamanında Özniteliklere Erişme

Öznitelikler (**attribute**) tek başına bir iş yapmaz. Onların içindeki bilgiyi okumak için yansıma (**reflection**) kütüphanesini kullanırız.

```
var tip = typeof(VeriIsleyici);
var oznitelikler = tip.GetCustomAttributes(typeof(BilgiAttribute), false);

foreach (BilgiAttribute attr in oznitelikler)
{
    Console.WriteLine($"Kod Yazarı: {attr.Yazar}, Versiyon: {attr.Versiyon}");
}
```

§11.4.4 Öznitelikler Nerede Kullanılır?

- ♦ **Veritabanı Eşleme:** İleride göreceğimiz varlık çerçevesinde, sınıftaki bir özelliğin veritabanındaki hangi tablo sütununa karşılık geleceğini belirtmek için `[Column("MusteriAdi")]` gibi nesne eşlemesinde ORM (**Object-Relational Mapping**) kullanılır.
- ♦ **Birim Testleri (unit test):** Bir yöntemin test yöntemi olduğunu işaretlemek için `[Test]` şeklinde kullanılır.
- ♦ **JSON İşlemleri:** Sınıftaki bir özelliğin, JSON (**Javascript Object Notation**) içindeki ismini değiştirmek için `[JsonPropertyName("user_id")]` şeklinde kullanılır.

Öznitelik

Bir sınıfa ek bilgi eklemek için neden kalıtım (**inheritance**) yerine öznitelik kullanmayı tercih ederiz? Kalıtım, bir "biridir" (**is-a**) ilişkisi kurar ve sınıfın yapısını değiştirir. Oysa öznitelikler, sınıfa dışarıdan "etiket" yapıştırmak gibidir. Sınıfın hiyerarşisini bozmadan ona çalışma zamanında keşfedilebilecek özellikler kazandırır. Bu da "gevşek bağıklık" (**loose coupling**) sağlar.

11.5 Yansıma

Yansıma (**reflection**), `System.Reflection` isim uzayı altında bulunur. Derlenmiş olan sınıf kütüphaneleri `*.dll` veya `.NET` uygulaması `*.exe` gibi montaj (**assembly**) dosyalarınızın içindeki tip bilgilerine ulaşmanızı sağlar.

§11.5.1 Temel Giriş Noktası Type Sınıfı

Yansıma dünyasında her şey `Type` sınıfı ile başlar. Bir sınıfın kimlik kartı gibidir. Bir tipin bilgilerine üç yolla ulaşabilirsiniz:

```
// 1. Tipin adından (Derleme zamanında biliniyorsa)
Type t1 = typeof(Kullanici);
```

```
// 2. Örnek bir nesneden
Kullanici k = new Kullanici();
Type t2 = k.GetType();
// 3. String bir isimden (Dinamik yükleme için ideal)
Type t3 = Type.GetType("Sistem.Modeller.Kullanici");
```

§11.5.2 Çalışma Zamanında Sınıfı Keşfetmek

Yansıma kullanarak bir sınıfın içindeki tüm detayları (`private` olanlar dahil!) dönebiliriz. Ancak **vaktinden önce derleme** AOT (**Ahead of Time Compilation**) gibi durumlarda yansıma işlemleri kısıtlanabilir. Vaktinden önce derleme 3.4 başlığında anlatılmıştı.

```
Type t = typeof(Kullanici);
Console.WriteLine($"Sınıf Adı: {t.Name}");
// Yöntemleri (method) listele
foreach (var method in t.GetMethods())
{
    Console.WriteLine($"Metot: {method.Name} - Dönüş Tipi: {method.ReturnType}");
}
// Özellikleri (property) listele
foreach (var prop in t.GetProperties())
{
    Console.WriteLine($"Özellik: {prop.Name}");
}
```

§11.5.3 Dinamik Nesne Oluşturma ve Yöntem Çağırma

Yansımanın asıl gücü, derleme zamanında (**compile-time**) adını bile bilmediğiniz bir sınıftan nesne üretip onun yöntemini çalıştırmaktır. Ama böyle bir sınıfı, kodunuzun erişebileceği bir noktada (aynı proje veya referans verilmiş bir `*.dll`) olarak tanımlamalısınız.

```
Type t = typeof(HesapMakinesi);

// 1. Dinamik olarak nesne örneği oluştur (Instantiate)
object instance = Activator.CreateInstance(t);

// 2. "Topla" metodunu bul
MethodInfo method = t.GetMethod("Topla"); //Yöntem bulamaz ve geriye "null" döndürür. Aramada
→ küçük büyük harf ayrımı yapıldığı unutulmamalıdır. Bu yüzden Yansıma kodlarında her zaman
→ null kontrolü yapmak profesyonel bir yaklaşımdır.

// 3. Metodu çalıştır (Invoke)
object[] parametreler = { 5, 10 };
object sonuc = method.Invoke(instance, parametreler);

Console.WriteLine($"Sonuç: {sonuc}"); // Çıktı: 15
```

§11.5.4 Mahrem Alanlara Müdahale

Yansıma, normal şartlarda erişemeyeceğiniz `private` alanlara bile erişebilir. Böylece kapsüllemeyi (**encapsulation**) delmiş oluruz. Bu, genellikle test araçları veya özel çerçeve (**framework**) yazımlarında kullanılır.

```
FieldInfo field = t.GetField("_gizliSifre", BindingFlags.NonPublic | BindingFlags.Instance);
field.SetValue(instance, "123456"); // Private alanı dışarıdan değiştirdik!
```

§11.5.5 Yansıma Nerede Kullanılır?

- ◊ **ORM Araçları:** Sınıf isimlerinizden tablo isimlerini, özellik isimlerinizden sütun isimlerini yansıma (**reflection**) kullanılarak eşleştirilir.
- ◊ **Bağımlılık Enjeksiyonu:** Yapıcılar (**constructor**) parametrelerini analiz ederek hangi sınıftan nesne üretmesi gerektiğini yansıma ile anlar.
- ◊ **JSON Serileştiriciler:** `Newtonsoft.Json`, nesnenin içindeki alanları tarayıp onları metin formatına yansıma ile dönüştürür.
- ◊ **Eklenti Sistemleri:** Programınız çalışırken dışarıdan bir `*.dll` dosyasını tarayıp içindeki uygun sınıfları yüklemek için yansıma kullanılır. Bu, eklenti (**plugin**) için uygun bir yöntemdir

Yansımanın Maliyeti

Yansıma (**reflection**) neden maliyetli bir işlemdir? Normal bir yöntem çağrısı derleme zamanında adreslenir ve çok hızlıdır. Ancak yansımada; **string** üzerinden yöntemi arar, güvenlik kontrollerini yapar, parametre tiplerini kontrol eder ve ardından çalıştırır. Bu da normal bir çağrıdan binlerce kat daha yavaş olabilir. Yansımayı sık çalışan döngülerin içinde kullanmaktan kaçınmalıyız veya sonuçları "önbelleğe" (**cache**) almalıyız.

2026 yılı itibarıyla modern .NET dünyasında yansımanın (**reflection**) en büyük alternatifi **kaynak üreticileri** (**source generators**) teknolojisidir. Yansımanın, çalışma zamanındaki (**run-time**) maliyetine karşın, kaynak üreticilerinin derleme zamanında (**compile-time**) kod üreterek "sıfır maliyetli" bir alternatif sunarlar.

11.6 Düşün ve Çöz

- ◇ **Düşün:** Yapı ile sınıf arasındaki bellek yönetimi farkı nedir? Bir yapı stack'te, bir sınıf heap'te bulunur. Bu ayrımın performans açısından ne anlamı vardır? Hangi durumlarda sınıf yerine yapı tercih edilmelidir?
- ◇ **Düşün:** Değer kopyalamanın (**value copy**) yapılar için yarattığı tuzaklar nelerdir? Bir yapı içindeki referans tipi alanlar kopyalandığında ne olur? Bu durum 'savunmacı kopyalama' (**defensive copy**) gerekliliğini nasıl doğurur?
- ◇ **Düşün:** C#10 ile gelen kayıt yapıları (**record struct**) ne zaman düz **struct**'a tercih edilir? Değiştirilemezlik (**immutability**) ile **record struct**'ın nasıl bir ilişkisi vardır?
- ◇ **Düşün:** Öznitelik (**attribute**) mekanizması çalışma zamanında ve derleme zamanında nasıl farklı kullanılır? [**Obsolete**], [**Serializable**] ve özel özniteliklerin bir framework veya araç için nasıl anlam ifade ettiğini açıklayınız.
- ◇ **Düşün:** Yansıma (**reflection**) nedir ve çalışma zamanında tip bilgisine erişmenin pratik kullanım alanları nelerdir? Yansımanın performans maliyeti var mıdır ve bu maliyet hangi senaryolarda kabul edilemez hale gelir?
- ◇ **Düşün:** Birlik (**union**) benzeri yapılar C#'ta nasıl oluşturulabilir? [**StructLayout(LayoutKind.Explicit)**] özniteliği ne işe yarar? Bu yapının birlikte çalışabilirlik (**interop**) senaryolarındaki önemi nedir?
- ◇ **Düşün:** **readonly struct** ve **in** parametresi anahtar kelimeleri performans optimizasyonunda nasıl kullanılır? Bu özelliklerin olmadığı durumda yapı kopyalamanın yarattığı maliyet nedir?
- ◇ **Çöz:** 2D koordinatı temsil eden Nokta adında bir **struct** tanımlayınız. İki nokta arasındaki uzaklığı hesaplayan, toplamı döndüren ve eşitliği kontrol eden yöntemler ekleyiniz. Aynı işlevi bir class ile uygulayın ve bellek davranışını tartışın.
- ◇ **Çöz:** Özel bir [**DoğrulamaKuralı**] özniteliği tasarlayın: **string** özellikler üzerinde minimum/maksimum uzunluk kuralları belirlenebilsin. Yansıma kullanarak bu kuralları çalışma zamanında denetleyen bir **Doğrulamayıcı** sınıfı yazınız.
- ◇ **Çöz:** Yansıma kullanarak çalışma zamanında bir sınıfın tüm özelliklerini, yöntemlerini ve alanlarını listeleyen bir 'nesne inceleyici' programı yazınız. Çıktıyı okunabilir biçimde formatlayın.
- ◇ **Çöz:** Bir nesneyi anahtar-değer çiftlerine (sözlük) ve geri çeviren (sözlükten nesneye) dönüştüren; yansımayı kullanan genel bir yardımcı sınıf yazınız.
- ◇ **Çöz:** [**JsonIgnore**] benzeri özel bir [**GizliAlan**] özniteliği oluşturunuz. Yansıma kullanarak bu öznitelikle işaretlenmiş alanları atlayan basit bir nesne serileştirici yazınız.